LeNet 加速器微架構說明書

Microarchitecture Specification　版本 1.8 精簡版

本設計用同一組 25 個 signed 8-bit 乘法器完成五層運算。CONV1 / CONV2 以 5×5 視窗計算；CONV3 / FC1 / FC2 每組最多算 8 筆，再累加成一個輸出。本文保留已確認架構，合併重複解釋，供 RTL 設計與驗證使用。

1 架構與資料流總覽

層	Dataflow 資料流	資料如何重用
CONV1	Row-Stationary-inspired + Weight Stationary	六列輸入暫存重用相鄰視窗；25 個權重固定在本地；只有一個輸入通道。
CONV2	Row-Stationary-inspired + Weight Stationary	沿用列重用與固定權重；2×10×32-bit 部分和累加六個輸入通道。
CONV3	Output Stationary（OS）	同一個部分和暫存器累加 400 個乘積，分 50 組。
FC1	Output Stationary（OS）	同一個部分和暫存器累加 120 個乘積，分 15 組。
FC2	Output Stationary（OS）	累加 84 個乘積：10 組各 8 筆，最後一組 4 筆。

Stationary 指資料在一段計算期間留在本地：Weight Stationary 留住目前 kernel 的權重；Output Stationary 留住目前輸出的部分和位置，數值仍持續更新。Row-Stationary-inspired 表示採用列資料重用的想法，不宣稱完整實作某種典型陣列。

只記住這幾個名稱

act：輸入值；weight：權重；row_sum：一列的加總；channel_sum：一個輸入通道的加總；conv_out：所有輸入通道加完的卷積值；psum：尚未加完的部分和。PE 是處理單元，每個含 5 個乘法器。SRAM 是儲存輸入、權重與輸出的記憶體。

Spatial / Vector 是硬體運算模式，dataflow 是資料保留與搬移方式，兩者描述不同面向。FC buffer 維持 120 bytes；FC1 完成後載回 84 筆給 FC2，不增加另一組 84-byte 暫存區。

2 共用核心與兩種運算模式

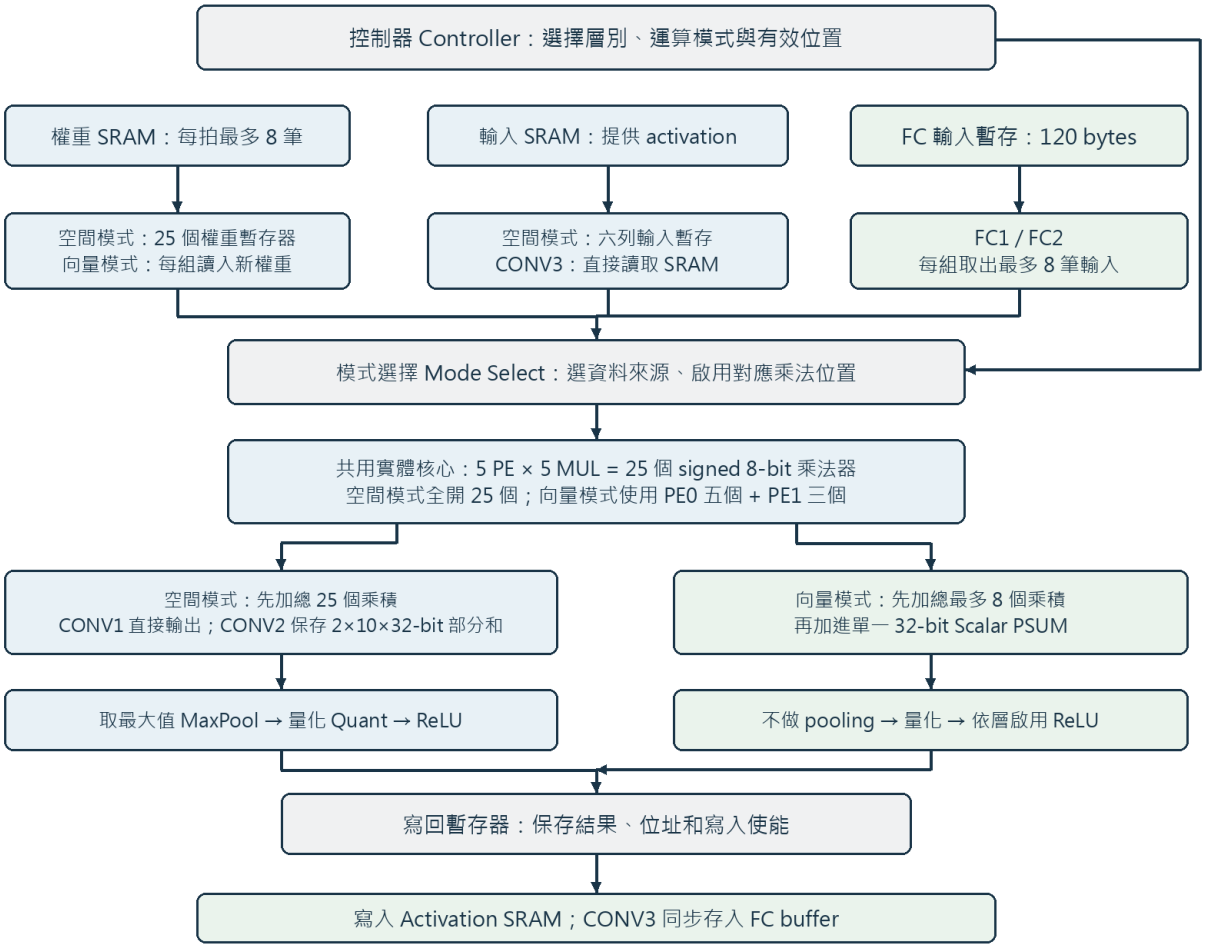


圖 1 同一組 25 個乘法器依層切換資料來源與加總路徑。

Spatial 啟用全部 25 個乘法器，從六列輸入暫存與 25 個權重暫存器取值。Vector 使用 PE0 五個、PE1 三個乘法位置，其餘停用；六列輸入暫存不使用，可停止更新或安全地關閉時脈。

CONV3 的 400 筆輸入由 Activation SRAM 讀取；FC1 / FC2 改讀本地 FC buffer。Vector 權重由雙埠 Weight SRAM 提供，兩埠各 32-bit，合計每拍最多 8 筆 8-bit 權重。

3 層級控制與位置編號

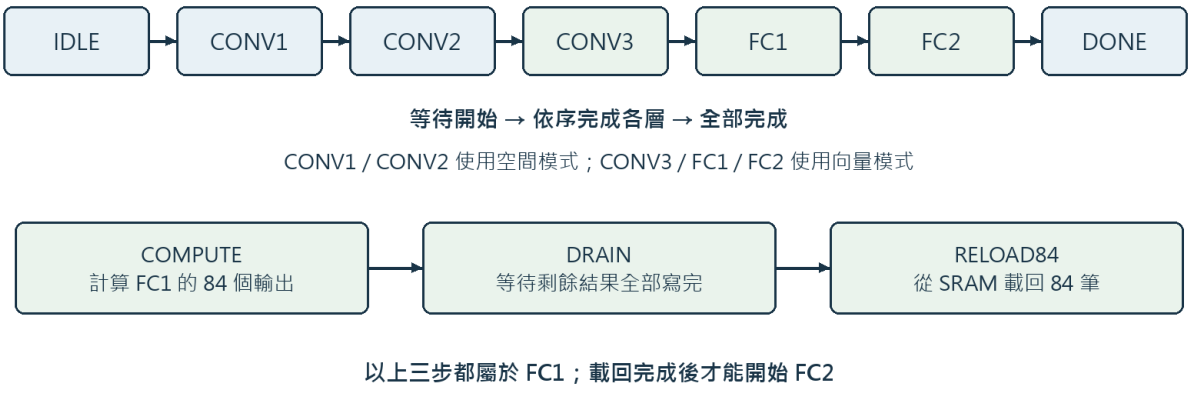


圖 2 外層決定執行哪一層；FC1 內另有計算、等待寫完、載回資料三個步驟。

strip 與 row_sel 一次看懂

strip 是同一輸出通道中的「相鄰兩列輸出區塊」；strip_idx 是區塊編號。row_sel 選區塊內的列：0 是上列、1 是下列；col 是 column（欄位）。idx 是 index（編號），sel 是 select（選擇），所有編號從 0 開始。

位置名稱	CONV1	CONV2	用途
out_ch	0 ~ 5	0 ~ 15	輸出通道編號
in_ch	0	0 ~ 5	輸入通道編號
strip_idx	0 ~ 13	0 ~ 4	第幾個兩列區塊
row_sel	0 或 1	0 或 1	區塊內上列或下列
col	0 ~ 27	0 ~ 9	pooling 前的輸出欄位

例如 strip_idx=3、row_sel=1，是整張輸出的第 7 列：out_row=2×strip_idx+row_sel=7。Vector 則用 neuron_idx（神經元編號）和 group_idx（輸入組別）計數。

切層前，要確認流水線、pooling 與最後一次 SRAM 寫入都完成；FC1 還要等 84 筆載回完畢。資料前進或停頓時，其層別、位置、組別與有效標記必須一起配對，避免結果寫錯位置。

4 乘法核心與三層加總

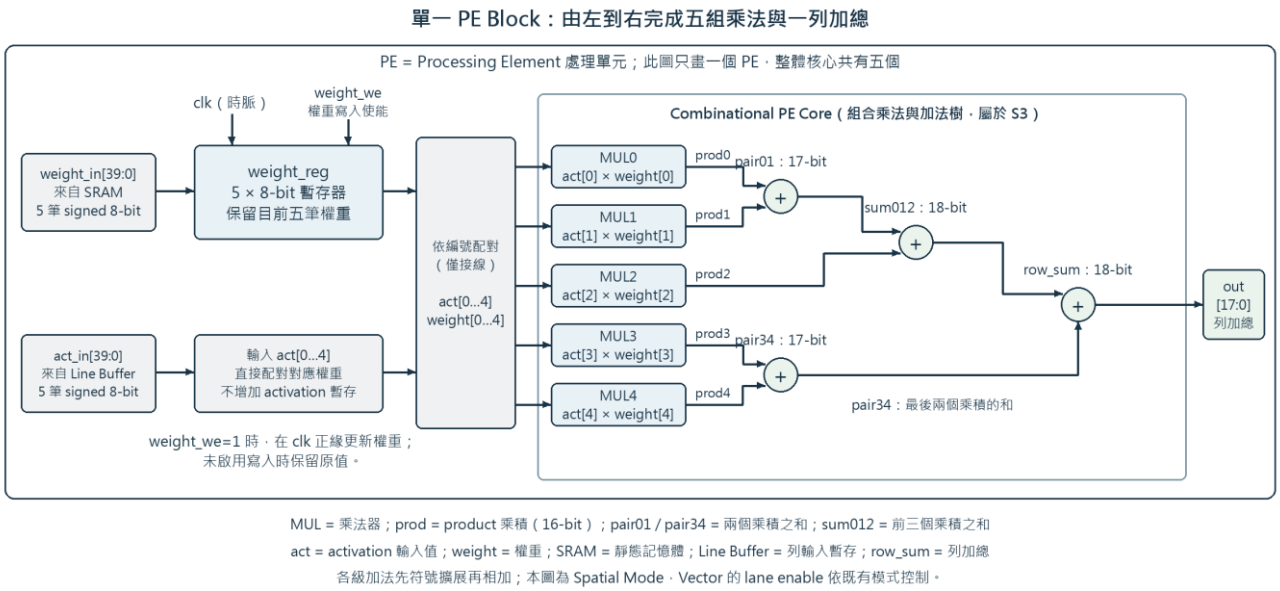


圖 3 單一 PE 的左至右資料流：權重暫存與輸入配對 → 五個乘法器 → 加法樹 → 18-bit 列加總。

一個 PE 保存五筆 8-bit 權重，與五筆輸入逐一相乘；五個 16-bit 乘積經加法樹合成 signed 18-bit row_sum。weight_we 只控制權重何時鎖入；乘法器與加法樹是 S3 的組合運算，不另增加一級管線。五個 PE 的 row_sum 再相加，得到 channel_sum。

步驟	做什麼	數字例子
row_sum 列加總	同一列五個輸入各乘對應權重，再相加。	輸入 [1,2,3,4,5]、權重 [2,-1,0,1,3] → 2-2+0+4+15=19。
channel_sum 單一通道加總	把五列的 row_sum 相加，得到一個輸入通道的 25 個乘積和。	五列 [19,5,-2,8,10] → 40。
conv_out 完整卷積結果	把各輸入通道的 channel_sum 相加。	Conv2 六通道 [40,10,-5,20,0,15] → 80。

上表是示意數字。Conv1 只有一個輸入通道，第二步就完成；Conv2 要加完六個通道才完成。conv_out 尚未包含另外定義的 bias、量化或 ReLU。詳細索引公式不再重複列出，RTL 以同一位置、同一組權重正確配對為準。

5 六列輸入暫存與兩列區塊排程

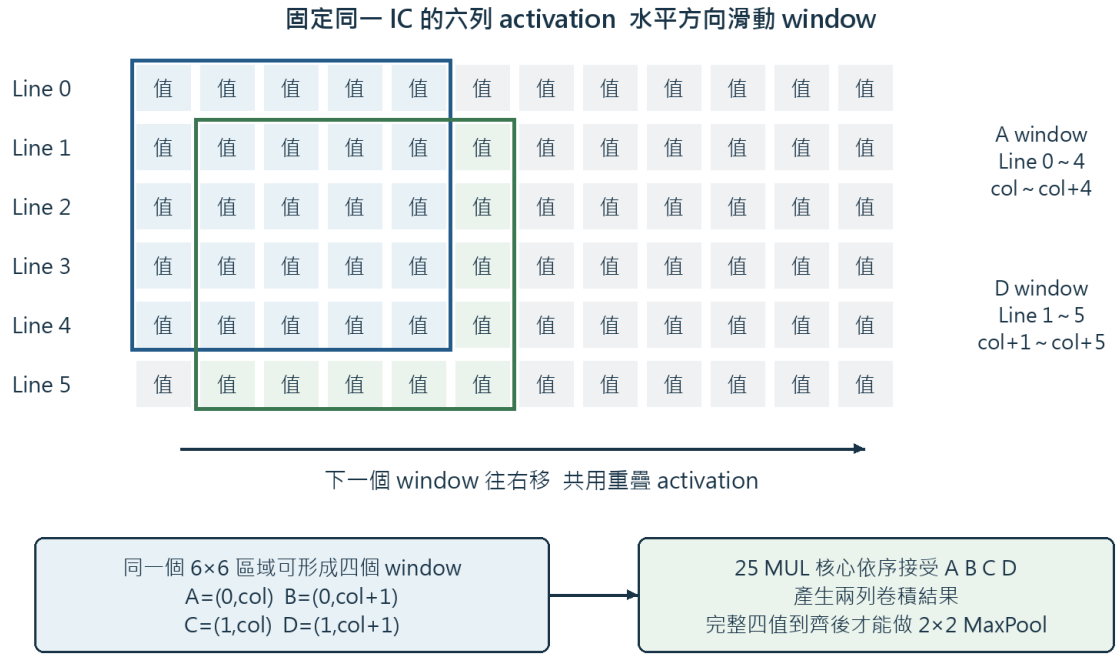


圖 4 上列輸出需輸入第 0～4 列，下列需第 1～5 列；合起來只需保留六列。

5×5 視窗向右移一欄會重用四欄，向下移一列會重用四列。這就是 Row-Stationary-inspired 的輸入重用；同一組 25 個權重則留在 Weight Reg，形成 Weight Stationary。

排程順序：固定輸出通道 → 選定兩列區塊 → 讓一個輸入通道掃完整個區塊 → 換下一個輸入通道。換通道時載入對應輸入與權重；現有六列暫存不代表同時保存所有輸入通道。

區塊內先掃上列再掃下列，或上下交錯，仍待 RTL 選定；這會影響 pooling 暫存量。若每列存 32 筆 8-bit 輸入，六列容量為 6×32×8=1,536 bits；實際列深度與讀取方式仍須確認。

6 CONV1 詳解

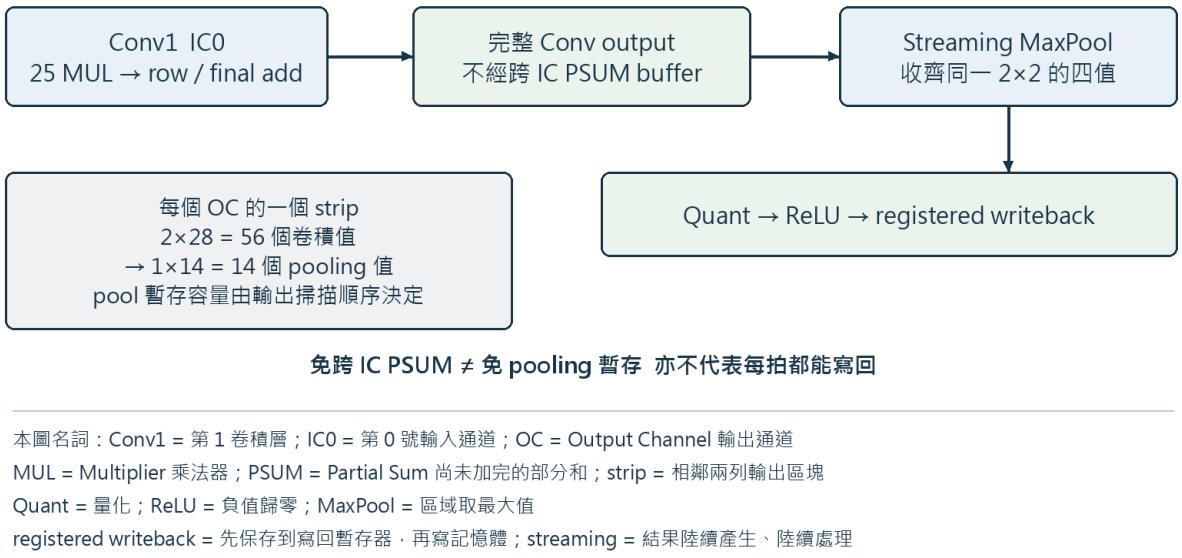


圖 5 CONV1 不需跨通道部分和；算出的完整值陸續送到 MaxPool。

資料流與計算

CONV1 採 Row-Stationary-inspired + Weight Stationary：六列暫存重用重疊輸入，25 個權重暫存器保留目前輸出通道的 5×5 kernel，供多個位置反覆使用。換輸出通道時才換對應權重；載入仍需時間。

輸入為 1×32×32，卷積輸出為 6×28×28。每個位置只需一次 25 個乘積的加總，因為輸入通道數只有 1，不需跨通道 PSUM。

Pooling 與寫回

Streaming MaxPool 表示結果陸續產生、陸續交給 pooling，不必先存完整特徵圖。但同一個 2×2 區域仍要收齊四個值，才能選最大值；尚未配對的值需要暫存。

每個兩列區塊有 2×28 個卷積值，pooling 後得到 14 筆。整層經 2×2、步長 2 的 pooling 後為 6×14×14，再量化、ReLU，經寫回暫存器寫 SRAM。

本層檢查重點

確認視窗邊界與 pooling 四值位置正確；不寫跨通道 PSUM；換輸出通道前，尚未使用完的輸入或結果不得被覆蓋。

7 CONV2 詳解

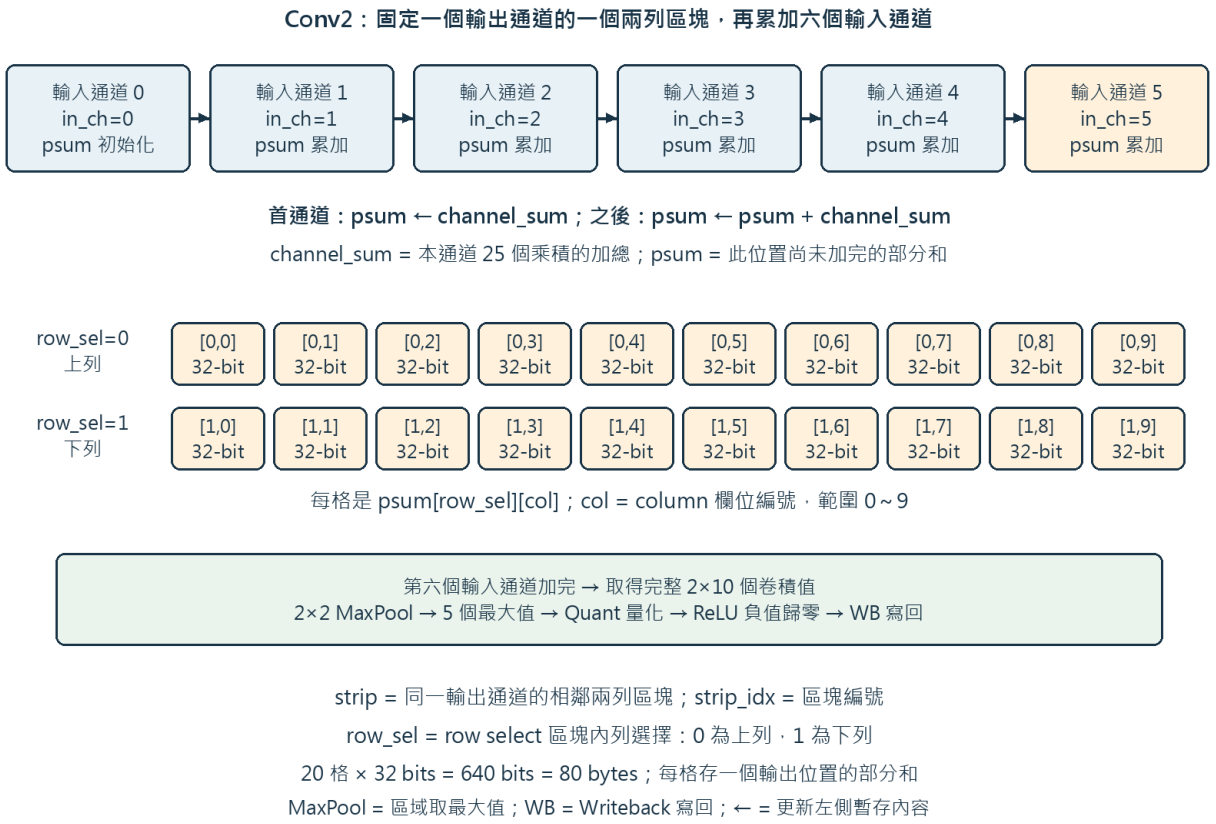


圖 6 每格保存一個位置的部分和；六個輸入通道全部加完後才做 pooling。

CONV2 同樣採 Row-Stationary-inspired + Weight Stationary。每個位置需累加六個輸入通道；目前通道的 25 個權重在掃描區塊期間固定，換通道時更新，並非六通道共用同一組權重。

PSUM 為 2×10×32-bit，共 20 格、80 bytes。首通道令 psum←channel_sum；後續執行 psum←psum+channel_sum。← 表示更新左側儲存值。也可選擇先清零再累加，RTL 須固定其中一種初始化方式。

輸入為 6×14×14，卷積輸出為 16×10×10，pooling 後為 16×5×5。最後一次累加須把「加法後的新值」交給 pooling；每格結果用完後才能覆寫。例：舊值 150 加上最後貢獻 60，應送出 210。

8 四級管線與有效資料

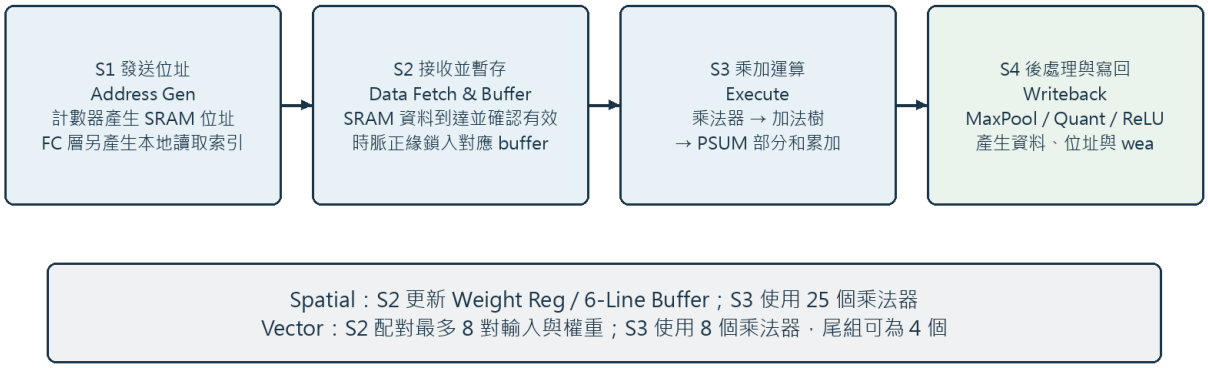


圖 7 SRAM 取數納入管線；乘法、加法樹與部分和累加全部屬於 S3。

階段	本級工作
S1 Address Gen 發送位址	產生 SRAM 位址；FC 層另外產生本地 buffer 索引。已暫存資料可以重用，不必每個視窗都重讀權重。
S2 Data Fetch & Buffer 接收並暫存	等 SRAM 回傳有效，再於時脈正緣鎖入相應暫存。Spatial 準備權重與輸入視窗；Vector 配對最多 8 對輸入與權重。
S3 Execute 乘加運算	執行乘法、加總與 PSUM 更新；Spatial 使用 25 個乘法位置，Vector 一般組 8 個、FC2 尾組 4 個。
S4 Writeback 後處理與寫回	完整結果才做 pooling / 量化 / 指定的 ReLU，再將資料、位址與寫入控制存入寫回暫存器。Vector 不做 pooling。

S2 未備妥就等待，不能讓 S3 使用未鎖存資料。S3 的乘法、加法與累加整段需通過時序檢查；若需要多拍，須停頓或正確傳遞最新部分和。S4 輸出暫存器不另列第五級，SRAM 實際接受時間仍依介面規定。

9 CONV3 FC1 FC2 的向量運算

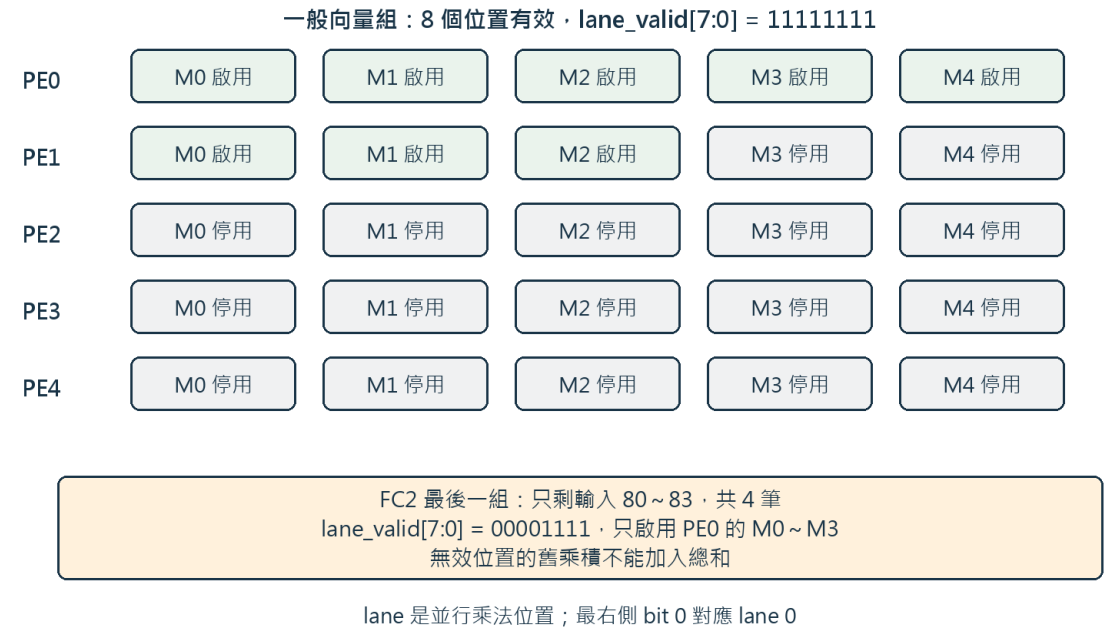


圖 8 一般組使用八個乘法位置；FC2 最後一組只使用四個。

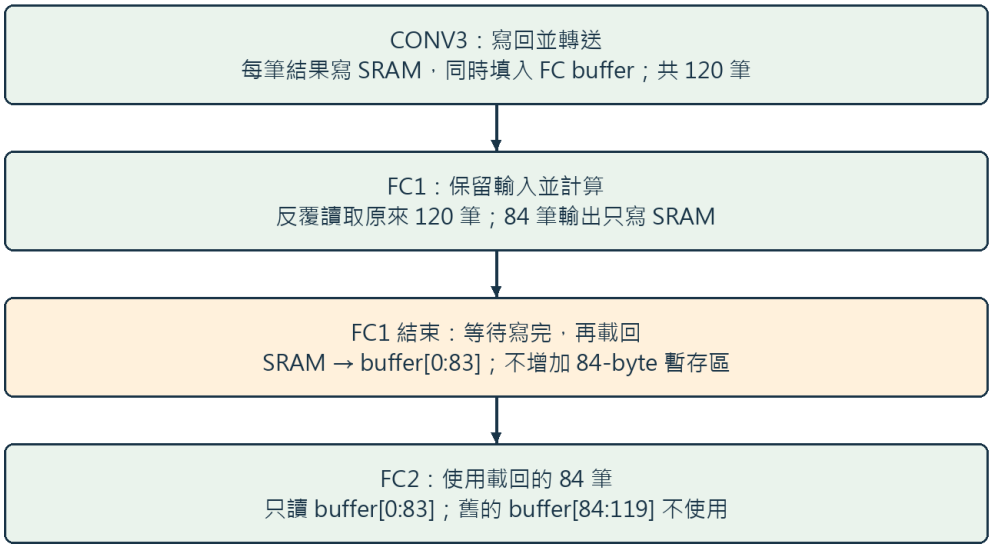
三層皆採 Output Stationary（OS）：本組乘積先相加成 group_sum，再加進同一個 scalar_psum，直到目前神經元完成。第一組初始化部分和，後續接續最新值；不是每個輸出各配置一個累加器。

層	每個輸出的計算	輸入來源與完成結果
CONV3	400 筆 = 50 組 × 8	Activation SRAM 提供 16×5×5；完成 120 筆輸出後，寫 SRAM 並轉送 FC buffer。
FC1	120 筆 = 15 組 × 8	反覆讀 FC buffer 的 120 筆；完成 84 筆輸出，只寫 SRAM。
FC2	84 筆 = 10 組 × 8 + 4	讀 FC buffer 前 84 筆；完成 10 筆輸出，依模型決定 ReLU。

valid 表示整筆工作有效；lane_valid 標記其中哪些乘法位置有效；last_group 表示該神經元的最後一組。一般組為 11111111，FC2 尾組為 00001111，最右邊是 lane 0。無效位置的舊乘積不能加入總和。

有效標記本身不會省電，還要保持無效乘法位置的輸入不變、停用暫存更新，或使用安全的時脈閘控；不能以真的計算 0×weight 取代停用。

10 FC Buffer 更新與供料速度



本圖名詞：CONV3 = 第 3 卷積層；FC1 / FC2 = 第 1 / 第 2 全連接層
SRAM = 靜態隨機存取記憶體；FC buffer = 全連接層輸入暫存區
buffer[0:83] = 編號 0～83 的 84 格；buffer[84:119] = 編號 84～119 的舊資料格

圖 9 保留單一 120-byte FC buffer；FC1 計算期間不覆寫原輸入。

每個 FC1 神經元都要讀完整 120 筆輸入。若第一個輸出立即覆寫 buffer[0]，後續神經元就讀不到原輸入。因此 FC1 的 84 筆輸出先只寫 SRAM；全部寫完後再載回 buffer[0:83]，才啟動 FC2。

CONV3 → FC1 的 write-and-forward（寫回同時轉送）省去一次輸入重讀；FC1 → FC2 仍需載回 84 筆。若 Activation SRAM 每拍可讀 8 bytes，需 11 次傳輸；若每拍 4 bytes，需 21 次。兩者都另計等待回應與控制時間。

CONV3 每組同時需要 8 筆輸入與 8 筆權重。若權重已到齊但輸入未齊，必須等待（stall）；不能只看 Weight SRAM 頻寬就保證每拍一組。FC1 / FC2 的本地 buffer 也要能一次提供最多 8 筆。

11 各層硬體分工與工作量

層與尺寸	模式與乘法器	輸入與權重來源
CONV1 1×32×32 → 6×28×28	Spatial；25 個	Activation SRAM → 六列暫存；25 Weight Reg 重用目前 kernel。
CONV2 6×14×14 → 16×10×10	Spatial；25 個	Activation SRAM → 六列暫存；換輸入通道時換 kernel。
CONV3 16×5×5 → 120	Vector；8 個	輸入與權重均由 SRAM 供應；每個輸出重新讀其所需輸入。
FC1 120 → 84	Vector；8 個	FC buffer[0:119] 提供輸入；Weight SRAM 串流。
FC2 84 → 10	Vector；8 個，尾組 4	FC buffer[0:83] 提供輸入；Weight SRAM 串流。

層	PSUM 與輸出處理	整層工作量
CONV1	不跨通道累加；pooling → 量化 / ReLU → 寫回。	4,704 個視窗；117,600 次乘法。
CONV2	2×10 Strip PSUM；六通道完成後 pooling → 量化 / ReLU → 寫回。	9,600 個單通道視窗；240,000 次乘法。
CONV3	單一 Scalar PSUM；量化 / ReLU → 寫回並轉送。	6,000 組；48,000 次乘法。
FC1	單一 Scalar PSUM；量化 / ReLU → 寫回；全層後載回 84 筆。	1,260 組；10,080 次乘法。
FC2	單一 Scalar PSUM；量化 / 模型指定 ReLU → 寫回。	110 組；840 次乘法。

尺寸依 5×5、步長 1、無 padding 的卷積，以及 2×2、步長 2 的 pooling。運算組數不是總週期：還要加 SRAM 等待、載入、管線起始與收尾、後處理、寫回和 RELOAD84。AT（面積×執行時間）以綜合與模擬量測為準。

12 寫回 數值與索引規則

寫回介面

結果先存入寫回暫存器，再送 SRAM。wb_valid 表示有有效結果；wb_addr 是目的位址；wb_data 是資料；wb_wea 指定允許寫入的位置。若 SRAM 暫時拒收（backpressure），這些欄位須一起保持，直到接受。CONV3 轉送 FC buffer 也必須對應同一筆有效提交。

Packing 是把多筆 8-bit 結果合成一個較寬的資料字，例如四筆合成 32-bit。未滿一個資料字時，寫入 mask 只能啟用有效 byte；實際 bus 寬、位址單位與封裝順序仍待定。

數值處理

加總範圍	最低帶符號位寬	本版處理
單一 8×8-bit 乘積	16-bit	乘法兩端均解讀為 signed。
一列 5 個乘積	18-bit	加法前先擴位。
一個視窗 25 個乘積	20-bit	不可先截成 16-bit 再加。
CONV2 的 150 個乘積	23-bit	Strip PSUM 使用 32-bit。
CONV3 的 400 個乘積	24-bit	Scalar PSUM 使用 32-bit。

以上不含 bias 或縮放。符號擴展是在增加位寬時保留正負號；量化則依模型縮放、捨入及飽和到 8-bit，不是直接截位。ReLU 將負值改成 0。MaxPool 使用完整值比較；改變量化與其他步驟的順序前，必須核對軟體參考模型。

只保留必要的索引例子

向量組：act_idx=8×group_idx+lane_idx。例：第 2 組、第 3 個位置 → 16+3=19。group_idx 是組別，lane_idx 是組內位置，act_idx 是輸入索引。

CONV3 展平：act_idx=25×in_ch+5×in_row+in_col。例：通道 2、列 1、欄 3 → 50+5+3=58。輸入與權重要採同一順序。FC2 每個神經元 84 bytes，32-bit word 對齊但不一定在 8-byte 邊界；讀取電路須選正確 word，尾組只用四筆。

13 儲存成本 待定項目與驗證

儲存	容量	用途
Weight Reg	25×8-bit = 25 bytes	Spatial 的目前 kernel。
Strip PSUM	2×10×32-bit = 80 bytes	CONV2 保存兩列部分和。
Scalar PSUM	1×32-bit = 4 bytes	三個 Vector 層共用。
FC_Act_Buffer	120×8-bit = 120 bytes	FC1 用 120 格；FC2 用前 84 格。
6-Line Buffer	6×LINE_DEPTH×8 bits	LINE_DEPTH 是每列筆數；Vector 不使用。

前四項合計 229 bytes，不含六列暫存、pipeline、pooling、寫回與控制。相較原 Spatial 架構，新增 124 bytes，沒有 84-byte staging。容量不等於面積，還要計入選擇電路與實作方式。

已定案與待確認

已定案：各層 dataflow、25 個實體乘法器、Vector 8 / 4 lanes、兩種 PSUM、120-byte FC buffer、RELOAD84 與四級管線。待確認：SRAM latency / 位址配置 / 共用資源、Activation SRAM 頻寬、line buffer 分區、PSUM 讀改寫、區塊內掃描與 pooling 暫存、量化 / bias、reset / start / done / stall 協定及目標時脈。

驗證時至少檢查這四組情境

Spatial：核對視窗邊界與 pooling 配對；CONV1 不做跨通道累加；CONV2 加完六通道才輸出，使用最後一次加法後的新值。

Vector：比對 400 / 120 / 84 筆內積；測試正負極值、第一組初始化、最後組輸出與神經元切換。FC2 尾組不讀越界資料，也不加入無效位置的舊乘積。

Buffer：CONV3 寫回與轉送一致；FC1 運算期間原輸入不變；全層寫完才載回，未滿 84 筆不得開始 FC2；FC2 不讀第 84 ~ 119 格。

控制與寫回：插入空拍、SRAM 等待、拒收與 reset；確認資料和標記配對、PSUM 不重複累加、無效資料不寫 SRAM，切層前所有工作完成。時脈閘控需採平台支援的安全方式。